Playwright API Testing: Week 2 Notes (Review & Interview Prep)

Nov 4, 2026 · @Kiran

Status

Week 2 covered Restful-Booker CRUD with token auth, a typed client class, and four auth schemes. Several tasks are written but not yet seen passing, so this table separates verified work from the rest.

Area	Tasks	Status
Booker auth	A (get a token), B (wrong password)	B verified. A written but still skipped
Booker CRUD	C and D (create, read back), E (PUT), F (PATCH), G (delete), H (no token)	G verified, F reported passing. The rest are written but not seen passing
Types and client	I (interfaces), J (`BookingClient`), K (CRUD tests with cleanup)	I verified. J and K written, full run not yet seen
API key	12 (ReqRes)	Verified: valid key, no key and wrong key all observed
Basic auth	13 (httpbin)	Steps 2 to 4 verified. Step 5 parked
JWT	14 (DummyJSON)	Steps 1 and 2 written. Steps 3 to 6 pending
OAuth	15 (client credentials)	Mock server verified. Client class and tests parked
Data-driven negatives	9 (bad credentials)	Not started

Config and tooling

Most Week 2 failures came from configuration, not test code. Three separate projects keep each API's settings apart.

```
// playwright.config.ts
import 'dotenv/config';
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  projects: [
    { name: 'jsonplaceholder', testMatch: /posts.*\.spec\.ts/,
      use: { baseURL: 'https://jsonplaceholder.typicode.com' } },
    { name: 'booker', testMatch: /booker.*\.spec\.ts/,
      use: { baseURL: 'https://restful-booker.herokuapp.com',
             extraHTTPHeaders: { Accept: 'application/json' } } },
    { name: 'auth', testMatch: /auth\..*\.spec\.ts/ },
  ],
});
```

Rules learned

- A run labeled `[chromium]` means no named project applied, so `baseURL` was missing or wrong. The label should read `[booker]` or `[auth]`.
- `Invalid URL` means no host was available, because a path like `/booking` had no `baseURL`. A 404 means the request reached a server that lacked the route.
- Run from the project root, where `package.json` and `.env` live. Commands run from a subfolder miss the config.
- A file name argument is a regular expression, so use `booker.crud.spec.ts` or forward slashes. A Windows path with `\` returns "No tests found".
- `dotenv` loads `.env` into `process.env`. Import it once at the top of the config, and keep `.env` in `.gitignore`.
- `process` is built into Node. Types come from `@types/node`, and newer TypeScript may need `"types": ["node"]` in `tsconfig.json`.
- `tsconfig.json` belongs in the project root, not in `tests/src`. Use `"module": "commonjs"` without `verbatimModuleSyntax`, which rejects `export` on values in CommonJS.

- `test.use(...)` at the top of a file applies to every test in it. Scope it inside a `test.describe` block, or tests that expect no credentials will silently send them.

Command	Use
`npx playwright test --project=booker`	Run one project
`npx playwright test booker.crud.spec.ts --project=booker`	Run one file in one project
`npx tsc --noEmit`	Type-check without running
`npx playwright show-report`	Open the HTML report

Restful-Booker: auth and CRUD

Restful-Booker is a practice API that stores data, so a full create, read, update, delete cycle works. Each test creates its own booking, because the API is shared and IDs differ on every run.

Flow

1. `POST /auth` with `admin` / `password123` returns a `token`.
2. `POST /booking` returns `{ bookingid, booking }`. The payload is nested under `booking`, so compare `body.booking`, not `body`.
3. `GET /booking/:id` returns the booking directly, with no wrapper.
4. `PUT` replaces the whole record and `PATCH` changes only the fields sent. Both need the header `Cookie: token=<value>`.
5. `DELETE` also needs the token. Proof of deletion is a follow-up `GET` returning 404.

Rules for reliable tests

- Never hardcode a token or booking ID. Extract it from a response and assert its type, e.g. `typeof body.bookingid` is `'number'`.
- A status code alone proves little. After PUT, PATCH and DELETE, read the record back and compare the data.
- After a rejected request (no token), confirm the booking survived and is unchanged.
- A 404 body on this API is plain text, so calling `.json()` on it throws. Check the status first, or use `.text()`.

- Put the response text in the failure message: `expect(response.status(), await response.text()).toBe(200)`.
- Clean up created bookings with a valid token, in `afterEach` or a `finally` block.

Observed behavior vs standard

Endpoint	Observed	Standard
`POST /auth`, bad credentials	200 with `{"reason":"Bad credentials"}`	401
`POST /booking`	200	201
`PUT` or `DELETE` without a token	403	401
`DELETE /booking/:id`	201 with body `Created`	200 or 204

Assert the observed behavior so the suite is stable, and record the deviation as a defect or improvement. A 200 with an error in the body is a design flaw, because clients that trust `response.ok()` treat the failed login as a success.

Types and the BookingClient

Interfaces and a client class remove the repeated payloads and helpers, and they are the API-testing version of a Page Object.

```
// src/types.ts
export interface BookingDates { checkin: string; checkout: string }
export interface Booking {
  firstname: string; lastname: string; totalprice: number;
  depositpaid: boolean; bookingdates: BookingDates; additionalneeds?: string;
}
export interface CreateBookingResponse { bookingid: number; booking: Booking
}
export interface AuthResponse { token: string }
export function newBooking(): Booking { /* returns a fresh valid payload */ }
```

Interface rules

- Property names must match the JSON keys exactly: `bookingid`, not `bookingId`.
- Model the real response shape. `CreateBookingResponse` nests `booking: Booking`, which is the wrapper trap from Task C.

- `export` every interface and function other files import.
- `newBooking()` returns a new object each call, so one test cannot change another's data. Override one field with `{ ...newBooking(), totalprice: 999 }`.
- Types are not checked at run time. Playwright strips them, so use `npx tsc --noEmit` or the editor to see type errors.

Client design

```
export class BookingClient {
  private token?: string;
  constructor(private request: APIRequestContext) {}

  async getToken(): Promise<string> { /* fetch once, then reuse this.token */
}
  create(booking: Booking)  { return this.request.post('/booking', { data:
booking }); }
  get(id: number)          { return this.request.get(`/booking/${id}`); }
  async remove(id: number, withAuth = true) {
    return this.request.delete(`/booking/${id}`, { headers: await
this.authHeaders(withAuth) });
  }
}
```

- Methods return the raw `APIResponse`. Assertions stay in the tests, so negative tests can see a 4xx.
- Only setup code throws. `getToken()` throws on failure, because a failed login should stop the test with a clear message.
- `withAuth = false` sends no `Cookie` header, so the no-token tests still work.
- Use backticks and `${id}` to build URLs. A string like `"/{id}"` sends the literal text.
- Pass the real argument through. `create(booking)` once ignored its parameter and sent `newBooking()`.
- `await` goes on the call that returns a Promise: `await response.json()`, not `(await response).json()`.

Sharing the client across tests

```
let client: BookingClient;
const createdIds: number[] = [];

test.beforeEach(({ request }) => { client = new BookingClient(request); });
test.afterEach(async () => { for (const id of createdIds) await
client.remove(id); createdIds.length = 0; });
```

- `request` exists only inside a test or hook, so the client is created in `beforeEach`. Tests then use the shared `client` without building their own.
- A helper function outside a test receives `request` as a parameter and types it as `APIRequestContext`.
- Push every new `bookingid` into `createdIds` the moment it is created, so cleanup runs even when an assertion fails.
- A custom fixture (Week 3) replaces this `let` plus `beforeEach` pattern, so a test can ask for `client` as a parameter.

Authentication types

The four mechanisms on the job description each identify a caller differently, and each was tested against a real or mock service.

Scheme	Credential sent	Identifies	Practice service
API key	Header `x-api-key: <key>`	The calling application	ReqRes
Basic	Header `Authorization: Basic <base64(user:pass)>`	A user	httpbin
JWT / Bearer	Header `Authorization: Bearer <token>`	A user, with an expiry	DummyJSON
OAuth client credentials	Token from `POST /oauth/token`, then Bearer	An application	Local mock server

API key (ReqRes)

- Keep the key in `.env` and read `process.env.REQRES_API_KEY`. Never paste it into code, chat or screenshots.

- Observed: a valid key returns 200, no key also returns 200 on a limited keyless tier, and a wrong key returns 403 with `error: 'invalid_api_key'`.
- The status cannot tell a valid key from no key. Assert `body._meta.context`, which is `legacy_success` for a key and `keyless_success` without one.
- `_meta` is an undocumented metadata field, so note the risk in a comment. Do not assert on `variant` or `message`.
- The first version of the test stayed green with the header removed. Always add an assertion that can only pass when the credential was recognised.

Basic auth (httpbin)

- `/basic-auth/user/pass` sets which credentials the server expects. The `Authorization` header carries what the client sends.
- Build the header with `Buffer.from('user:pass').toString('base64')`, then prefix `Basic ` including the space.
- Playwright's `httpCredentials` option is the built-in alternative. By default it sends credentials only after a 401 challenge, and `send: 'always'` sends them on the first request.
- Wrong credentials and no credentials both returned 401 with `www-authenticate: Basic realm="Fake Realm"` and an empty body, so the server does not reveal whether a username exists.
- Basic auth is encoding, not encryption. Anyone who sees the header can decode it, so use it only over HTTPS.
- A password can contain a colon, so split the decoded text on the first colon only.

JWT / Bearer (DummyJSON)

- `POST /auth/login` with `expiresInMins` returns `accessToken` and `refreshToken`. `GET /auth/me` with the Bearer header returns the logged-in user.
- Assert `typeof body.accessToken` is `'string'`, not that the token equals the word `"string"`.
- A JWT has three dot-separated parts: header, payload, signature. The payload is readable by anyone, so never put secrets in it. The signature proves it was not altered.
- Read the user from the `/auth/me` response. Comparing the login response instead would pass even if `/auth/me` returned the wrong user.
- Do not print tokens. In CI logs they are stored and visible to others.

OAuth 2.0 client credentials (mock server)

- The token request is form-encoded, so use `form: { grant_type, client_id, client_secret }`, not `data`. A JSON body is rejected with `unsupported_grant_type`.
- The response holds `access_token`, `token_type: 'Bearer'` and `expires_in` in seconds.
- A client caches the token and its expiry. `Date.now()` is in milliseconds and `expires_in` is in seconds, so multiply by 1000. Treat the token as expired about one second early.
- The mock server starts in `beforeAll` and stops in `afterAll`, so it only exists while tests run. Use `test.describe.configure({ mode: 'serial' })` and compare `tokensIssued()` against a starting value.
- This token represents an application, not a person. Real providers differ in detail, and many expect the client credentials in a Basic auth header, so read the provider's documentation.

Code: Restful-Booker tests (Tasks A to H)

To practice, read the task, close the doc and write the test from memory. Then compare, and break each test once on purpose to prove it can fail. These versions use plain helper functions. Tasks I to K later in this doc rebuild the same tests with a client class.

```
// tests/booker.auth.spec.ts   (project: booker)
import { test, expect, APIRequestContext } from '@playwright/test';

const booking = {
  firstname: 'Jim',
  lastname: 'Brown',
  totalprice: 111,
  depositpaid: true,
  bookingdates: { checkin: '2018-01-01', checkout: '2019-01-01' },
};

async function getToken(request: APIRequestContext): Promise<string> {
  const res = await request.post('/auth', {
    data: { username: 'admin', password: 'password123' },
  });
  expect(res.status(), await res.text()).toBe(200);
  return (await res.json()).token;
}
```

```
async function createBooking(request: APIRequestContext, payload: object):
Promise<number> {
  const res = await request.post('/booking', { data: payload });
  expect(res.status(), await res.text()).toBe(200);
  return (await res.json()).bookingid;
}

// Deletes a booking this test created, so the shared API stays clean
async function cleanup(request: APIRequestContext, id: number) {
  const token = await getToken(request);
  await request.delete(`/booking/${id}`, { headers: { Cookie:
`token=${token}` } });
}
```

Task A: get a token

```
test('POST /auth returns a token for valid credentials', async ({ request })
=> {
  const response = await request.post('/auth', {
    data: { username: 'admin', password: 'password123' },
  });
  expect(response.status(), await response.text()).toBe(200);

  const body = await response.json();
  expect(typeof body.token).toBe('string');
  expect(body.token.length).toBeGreaterThan(0);
});
```

Task B: wrong password

```
test('POST /auth rejects a wrong password', async ({ request }) => {
  const response = await request.post('/auth', {
    data: { username: 'admin', password: 'wrong-password' },
  });
  const text = await response.text();

  // Observed: 200, though the standard would be 401. Logged as a deviation.
  expect(response.status(), text).toBe(200);

  const body = JSON.parse(text);
  expect(body).not.toHaveProperty('token');
  expect(body.reason, 'error reason should be returned').toBe('Bad
credentials');
});
```

Tasks C and D: create, then read back

```
test('POST /booking creates a booking that GET can read back', async ({
request }) => {
  const createResponse = await request.post('/booking', { data: booking });
  const created = await createResponse.json();

  // Observed: 200 (many APIs return 201 Created)
  expect(createResponse.status(), JSON.stringify(created)).toBe(200);
  expect(typeof created.bookingid).toBe('number');
  expect(created.booking).toMatchObject(booking);   // nested under `booking`

  const id: number = created.bookingid;
  try {
    const getResponse = await request.get(`/booking/${id}`);
    expect(getResponse.status(), await getResponse.text()).toBe(200);
    expect(await getResponse.json()).toMatchObject(booking);   // GET has no
wrapper
  } finally {
    await cleanup(request, id);
  }
});
```

Task E: update with PUT

```
test('PUT /booking/:id replaces a booking', async ({ request }) => {
  const id = await createBooking(request, booking);
  const token = await getToken(request);
  const updated = { ...booking, totalprice: 250 };

  try {
    const response = await request.put(`/booking/${id}`, {
      data: updated,
      headers: { Cookie: `token=${token}` },
    });
    expect(response.status(), await response.text()).toBe(200);
    expect(await response.json()).toMatchObject(updated);

    // Prove it was saved
    const getResponse = await request.get(`/booking/${id}`);
    expect(getResponse.status()).toBe(200);
    expect(await getResponse.json()).toMatchObject(updated);
  } finally {
    await cleanup(request, id);
  }
});
```

Task F: partial update with PATCH

```
test('PATCH /booking/:id changes only the given field', async ({ request })
=> {
  const id = await createBooking(request, booking);
  const token = await getToken(request);

  try {
    const response = await request.patch(`/booking/${id}`, {
      data: { firstname: 'Patched' },
      headers: { Cookie: `token=${token}` },
    });
    expect(response.status(), await response.text()).toBe(200);

    // One field changed, everything else untouched
    expect(await response.json()).toMatchObject({ ...booking, firstname:
'Patched' });
```

```
    const getResponse = await request.get(`/booking/${id}`);
    expect(await getResponse.json()).toMatchObject({ ...booking, firstname:
'Patched' });
  } finally {
    await cleanup(request, id);
  }
});
```

Task G: delete

```
test('DELETE /booking/:id removes a booking', async ({ request }) => {
  const id = await createBooking(request, booking);
  const token = await getToken(request);

  const response = await request.delete(`/booking/${id}`, {
    headers: { Cookie: `token=${token}` },
  });
  // Observed: 201 with body 'Created' (the standard would be 200 or 204)
  expect(response.status(), await response.text()).toBe(201);

  // Proof it is gone. The 404 body is plain text, so do not call .json()
  const getResponse = await request.get(`/booking/${id}`);
  expect(getResponse.status(), 'deleted booking should return
404').toBe(404);
});
```

Task H: requests without a token

```
test('DELETE /booking/:id without a token is rejected', async ({ request })
=> {
  const id = await createBooking(request, booking);
  try {
    // Observed: 403 (the standard for a missing token would be 401)
    const response = await request.delete(`/booking/${id}`);
    expect(response.status(), await response.text()).toBe(403);

    // Proof the booking survived
    const getResponse = await request.get(`/booking/${id}`);
    expect(getResponse.status()).toBe(200);
    expect(await getResponse.json()).toMatchObject(booking);
  } finally {
    await cleanup(request, id);
```

```
  }
});

test('PUT /booking/:id without a token is rejected', async ({ request }) => {
  const id = await createBooking(request, booking);
  try {
    const response = await request.put(`/booking/${id}`, {
      data: { ...booking, totalprice: 999 },
    });
    expect(response.status(), await response.text()).toBe(403);

    // Proof the data did not change
    const getResponse = await request.get(`/booking/${id}`);
    expect((await getResponse.json()).totalprice).toBe(111);
  } finally {
    await cleanup(request, id);
  }
});
```

Code: types, client and CRUD tests (Tasks I to K)

These files replace the helper functions above. Keep the client file free of `.spec` in its name, so the `booker` project does not treat it as a test.

tsconfig.json (project root)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "moduleResolution": "node",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true,
    "noUnusedParameters": true,
    "types": ["node"]
  },
  "include": ["tests", "src", "playwright.config.ts"]
}
```

Task I: src/types.ts

```
export interface BookingDates {
  checkin: string;
  checkout: string;
}

export interface Booking {
  firstname: string;
  lastname: string;
  totalprice: number;
  depositpaid: boolean;
  bookingdates: BookingDates;
  additionalneeds?: string;
}

export interface CreateBookingResponse {
  bookingid: number;   // lowercase, matching the JSON key exactly
  booking: Booking;    // the nested wrapper
}

export interface AuthResponse {
  token: string;
}

// A fresh object on every call, so one test cannot change another's data
export function newBooking(): Booking {
  return {
    firstname: 'Kiran',
    lastname: 'Test',
    totalprice: 150,
    depositpaid: true,
    bookingdates: { checkin: '2026-12-01', checkout: '2026-12-05' },
    additionalneeds: 'Breakfast',
  };
}
```

Check it with `npx tsc --noEmit`, then change `totalprice` to the string `'150'` and confirm TypeScript reports an error.

Task J: src/booking-client.ts

```
import { APIRequestContext, APIResponse } from '@playwright/test';
import { AuthResponse, Booking } from './types';

export class BookingClient {
  private token?: string;

  constructor(private request: APIRequestContext) {}

  // Setup helper: throws on failure so a broken login stops the test clearly
  async getToken(): Promise<string> {
    if (this.token) return this.token;   // reuse after the first login

    const response = await this.request.post('/auth', {
      data: { username: 'admin', password: 'password123' },
    });
    if (!response.ok()) {
      throw new Error(`Auth request failed: ${response.status()} ${await
response.text()}`);
    }

    const body: AuthResponse = await response.json();
    if (!body.token) {
      // This API returns 200 with { reason } for bad credentials
      throw new Error(`Auth returned no token: ${JSON.stringify(body)}`);
    }

    this.token = body.token;
    return this.token;
  }

  // Cookie header for protected endpoints, or none when a test skips auth
  private async authHeaders(withAuth: boolean): Promise<Record<string,
string>> {
    return withAuth ? { Cookie: `token=${await this.getToken()}` } : {};
  }

  // The methods below return the raw response. Assertions stay in the tests.
  create(booking: Booking): Promise<APIResponse> {
    return this.request.post('/booking', { data: booking });
  }
```

```
  get(id: number): Promise<APIResponse> {
    return this.request.get(`/booking/${id}`);
  }

  async update(id: number, booking: Booking, withAuth = true):
Promise<APIResponse> {
    return this.request.put(`/booking/${id}`, {
      data: booking,
      headers: await this.authHeaders(withAuth),
    });
  }

  async patch(id: number, fields: Partial<Booking>, withAuth = true):
Promise<APIResponse> {
    return this.request.patch(`/booking/${id}`, {
      data: fields,
      headers: await this.authHeaders(withAuth),
    });
  }

  async remove(id: number, withAuth = true): Promise<APIResponse> {
    return this.request.delete(`/booking/${id}`, {
      headers: await this.authHeaders(withAuth),
    });
  }
}
```

A quick smoke test of the client: create a booking, then `get` returns 200, `update` and `patch` return 200, `remove(id, false)` returns 403, `remove(id)` returns 201, and a final `get` returns 404.

Task K: tests/booker.crud.spec.ts

```
import { test, expect } from '@playwright/test';
import { BookingClient } from '../src/booking-client';
import { Booking, CreateBookingResponse, newBooking } from '../src/types';

let client: BookingClient;
const createdIds: number[] = [];

test.beforeEach(({ request }) => {
  client = new BookingClient(request);
```

```
});

test.afterEach(async () => {
  for (const id of createdIds) {
    await client.remove(id);   // harmless if the booking is already gone
  }
  createdIds.length = 0;
});

// Creates a booking, asserts it worked, and records the ID for cleanup
async function createTracked(booking: Booking = newBooking()):
Promise<CreateBookingResponse> {
  const response = await client.create(booking);
  expect(response.status(), await response.text()).toBe(200);
  const created: CreateBookingResponse = await response.json();
  createdIds.push(created.bookingid);
  return created;
}

test('POST /booking creates a booking', async () => {
  const booking = newBooking();
  const created = await createTracked(booking);

  expect(typeof created.bookingid).toBe('number');
  expect(created.booking).toMatchObject(booking);
});

test('GET /booking/:id returns the booking', async () => {
  const booking = newBooking();
  const { bookingid } = await createTracked(booking);

  const response = await client.get(bookingid);
  expect(response.status(), await response.text()).toBe(200);
  expect(await response.json()).toMatchObject(booking);
});

test('PUT /booking/:id replaces the booking', async () => {
  const booking = newBooking();
  const { bookingid } = await createTracked(booking);
  const updated = { ...booking, totalprice: 999 };

  const response = await client.update(bookingid, updated);
  expect(response.status(), await response.text()).toBe(200);
```

```
  expect(await response.json()).toMatchObject(updated);

  const readBack = await client.get(bookingid);
  expect(await readBack.json()).toMatchObject(updated);
});

test('PATCH /booking/:id changes only the given field', async () => {
  const booking = newBooking();
  const { bookingid } = await createTracked(booking);

  const response = await client.patch(bookingid, { firstname: 'Patched' });
  expect(response.status(), await response.text()).toBe(200);
  expect(await response.json()).toMatchObject({ ...booking, firstname:
'Patched' });
});

test('DELETE /booking/:id removes the booking', async () => {
  const { bookingid } = await createTracked();

  const response = await client.remove(bookingid);
  expect(response.status(), await response.text()).toBe(201);   // observed;
the standard is 200 or 204

  const readBack = await client.get(bookingid);
  expect(readBack.status(), 'deleted booking should return 404').toBe(404);
});

test('DELETE /booking/:id without a token is rejected', async () => {
  const booking = newBooking();
  const { bookingid } = await createTracked(booking);

  const response = await client.remove(bookingid, false);
  expect(response.status(), await response.text()).toBe(403);

  const readBack = await client.get(bookingid);
  expect(readBack.status()).toBe(200);
  expect(await readBack.json()).toMatchObject(booking);
});
```

Run it from the project root with `npx playwright test booker.crud.spec.ts --project=booker`. The output should start with `[booker]`.

Code: API key and Basic auth (Tasks 12 and 13)

Both files run in the `auth` project and use full URLs. The key lives in `.env` as `REQRES_API_KEY=...`, loaded by `import 'dotenv/config'` at the top of `playwright.config.ts`.

Task 12: tests/auth.apikey.spec.ts

```
import { test, expect } from '@playwright/test';

const apiKey = process.env.REQRES_API_KEY;
const url = 'https://reqres.in/api/users?page=2';

test('GET /api/users with a valid key is recognised', async ({ request }) =>
{
  test.skip(!apiKey, 'REQRES_API_KEY not set');

  const response = await request.get(url, { headers: { 'x-api-key': apiKey! }
});
  expect(response.status(), await response.text()).toBe(200);

  const body = await response.json();
  expect(body.page).toBe(2);
  expect(body.data.length).toBeGreaterThan(0);
  expect(body.data[0]).toHaveProperty('email');
  // The key was recognised, so this is not the keyless tier (undocumented
field)
  expect(body._meta.context, 'valid key should not be treated as
keyless').not.toBe('keyless_success');
});

test('GET /api/users with no key falls back to the keyless tier', async ({
request }) => {
  const response = await request.get(url);
  expect(response.status()).toBe(200);   // observed; the standard for a
missing key would be 401

  const body = await response.json();
  expect(body._meta.context).toBe('keyless_success');
});

test('GET /api/users with a wrong key is rejected', async ({ request }) => {
```

```
  const response = await request.get(url, { headers: { 'x-api-key': 'invalid-
key' } });
  expect(response.status(), await response.text()).toBe(403);   // standard
would usually be 401

  const body = await response.json();
  expect(body.error).toBe('invalid_api_key');
  expect(body).not.toHaveProperty('data');   // no user data leaks on a
rejected key
});
```

Break it once: rename the header to `x-api-keys` in the first test. The request becomes keyless, so the `not.toBe('keyless_success')` assertion fails.

Task 13: tests/auth.basic.spec.ts

There is no `test.use` at file level, because it would apply to every test and leak credentials into the no-credentials test.

```
import { test, expect } from '@playwright/test';

const url = 'https://httpbin.org/basic-auth/user/pass';

// Step 1: a manually built header. Keep this outside any describe that sets
httpCredentials.
test('GET /basic-auth with a manual Authorization header is authenticated',
async ({ request }) => {
  const encoded = Buffer.from('user:pass').toString('base64');

  const response = await request.get(url, {
    headers: { Authorization: `Basic ${encoded}` },
  });
  expect(response.status(), await response.text()).toBe(200);

  const body = await response.json();
  expect(body.authenticated).toBe(true);
  expect(body.user).toBe('user');
});

// Step 2: Playwright's built-in option
test.describe('httpCredentials, valid', () => {
  test.use({ httpCredentials: { username: 'user', password: 'pass' } });
```

```
  test('GET /basic-auth with httpCredentials is authenticated', async ({
request }) => {
    const response = await request.get(url);
    expect(response.status(), await response.text()).toBe(200);

    const body = await response.json();
    expect(body.authenticated).toBe(true);
    expect(body.user).toBe('user');
  });
});

// Step 3: wrong password
test.describe('httpCredentials, wrong password', () => {
  test.use({ httpCredentials: { username: 'user', password: 'wrong' } });

  test('GET /basic-auth with a wrong password is rejected', async ({ request
}) => {
    const response = await request.get(url);

    expect(response.status(), await response.text()).toBe(401);
    expect(response.headers()['www-authenticate']).toContain('Basic');
    expect(await response.text()).toBe('');
  });
});

// Step 4: no credentials. Same status, header and empty body as the wrong
password.
test('GET /basic-auth with no credentials is rejected', async ({ request })
=> {
  const response = await request.get(url);

  expect(response.status(), await response.text()).toBe(401);
  expect(response.headers()['www-authenticate']).toContain('Basic');
  expect(await response.text()).toBe('');
});

// Step 5: pure function test, no network
test('Basic auth header decodes back to the original credentials', () => {
  const encoded = Buffer.from('user:pass').toString('base64');
  const headerValue = `Basic ${encoded}`;

  expect(headerValue).toMatch(/^Basic [A-Za-z0-9+/]+=*$/);
```

```
  const [scheme, token] = headerValue.split(' ');
  expect(scheme).toBe('Basic');

  const decoded = Buffer.from(token, 'base64').toString();
  expect(decoded).toBe('user:pass');

  // Split on the FIRST colon only, because a password may contain colons
  const separator = decoded.indexOf(':');
  expect(decoded.slice(0, separator)).toBe('user');
  expect(decoded.slice(separator + 1)).toBe('pass');
});
```

Break each one once: change one character of the password in step 1, change `'wrong'` to `'pass'` in step 3 (it should then fail with 200 instead of 401), and add a file-level `test.use` with valid credentials to see step 4 fail.

Code: JWT and OAuth (Tasks 14 and 15)

Steps 1 and 2 of Task 14 are complete. Steps 3 to 6 are practice stubs: print the status and body first, then write the assertions for what you actually see. Task 15 is written but not yet run, so treat it as a starting point and print before you assert.

Task 14: tests/auth.jwt.spec.ts

```
import { test, expect, APIRequestContext } from '@playwright/test';

const base = 'https://dummyjson.com';

async function login(request: APIRequestContext) {
  const response = await request.post(`${base}/auth/login`, {
    data: { username: 'emilys', password: 'emilyspass', expiresInMins: 30 },
  });
  expect(response.status(), await response.text()).toBe(200);
  return response.json();
}

// Step 1
test('POST /auth/login returns access and refresh tokens', async ({ request
}) => {
  const body = await login(request);
```

```
  expect(typeof body.accessToken).toBe('string');
  expect(typeof body.refreshToken).toBe('string');
  expect(body.accessToken).toMatch(/^[\w-]+\.[\w-]+\.[\w-]+$/);   //
header.payload.signature
  expect(body.username).toBe('emilys');
});

// Step 2
test('GET /auth/me with a valid Bearer token returns the logged-in user',
async ({ request }) => {
  const loginBody = await login(request);

  const meResponse = await request.get(`${base}/auth/me`, {
    headers: { Authorization: `Bearer ${loginBody.accessToken}` },
  });
  expect(meResponse.status(), await meResponse.text()).toBe(200);

  const me = await meResponse.json();   // read the /auth/me body, not the
login body
  expect(me.username).toBe('emilys');
  expect(me.id).toBe(loginBody.id);
});

// Step 3: no token
test('GET /auth/me with no token is rejected', async ({ request }) => {
  const response = await request.get(`${base}/auth/me`);
  console.log(response.status(), await response.text());
  // TODO: assert the status and error message you observe
});

// Step 4: garbage token
test('GET /auth/me with a garbage token is rejected', async ({ request }) =>
{
  const response = await request.get(`${base}/auth/me`, {
    headers: { Authorization: 'Bearer not-a-real-token' },
  });
  console.log(response.status(), await response.text());
  // TODO: assert the status and error message you observe
});

// Step 5: tampered signature
test('GET /auth/me with a tampered token is rejected', async ({ request }) =>
{
```

```
  const { accessToken } = await login(request);
  const last = accessToken.slice(-1);
  const tampered = accessToken.slice(0, -1) + (last === 'a' ? 'b' : 'a');
// change one character

  const response = await request.get(`${base}/auth/me`, {
    headers: { Authorization: `Bearer ${tampered}` },
  });
  console.log(response.status(), await response.text());
  // TODO: assert the status and error message you observe
});

// Step 6: wrong password
test('POST /auth/login with a wrong password is rejected', async ({ request
}) => {
  const response = await request.post(`${base}/auth/login`, {
    data: { username: 'emilys', password: 'wrong', expiresInMins: 30 },
  });
  console.log(response.status(), await response.text());
  // TODO: assert the status, and that the body has no accessToken
});
```

Break it once: change `Bearer` to `Basic` in step 2. Do not print the tokens themselves.

Task 15: the mock server (tests/helpers/mock-oauth-server.ts)

```
import http from 'http';

export function createMockOAuth(port = 4010, ttlSeconds = 3) {
  const tokens = new Map<string, number>();
  let issued = 0;

  const server = http.createServer((req, res) => {
    let raw = '';
    req.on('data', (chunk) => (raw += chunk));
    req.on('end', () => {
      res.setHeader('Content-Type', 'application/json');

      if (req.method === 'POST' && req.url === '/oauth/token') {
        const p = new URLSearchParams(raw);
        if (p.get('grant_type') !== 'client_credentials') {
          res.statusCode = 400;
          return res.end(JSON.stringify({ error: 'unsupported_grant_type'
```

```
}));
        }
        if (p.get('client_id') !== 'my-client' || p.get('client_secret') !==
'my-secret') {
          res.statusCode = 401;
          return res.end(JSON.stringify({ error: 'invalid_client' }));
        }
        const token = `token-${++issued}`;
        tokens.set(token, Date.now() + ttlSeconds * 1000);
        return res.end(JSON.stringify({ access_token: token, token_type:
'Bearer', expires_in: ttlSeconds }));
      }

      if (req.method === 'GET' && req.url === '/api/data') {
        const token = (req.headers.authorization ?? '').replace('Bearer ',
'');
        const expiry = tokens.get(token);
        if (!expiry || expiry < Date.now()) {
          res.statusCode = 401;
          return res.end(JSON.stringify({ error: 'invalid_token' }));
        }
        return res.end(JSON.stringify({ data: 'secret' }));
      }

      res.statusCode = 404;
      res.end('{}');
    });
  });

  return {
    start: () => new Promise<void>((resolve) => server.listen(port,
resolve)),
    stop: () =>
      new Promise<void>((resolve) => {
        server.closeAllConnections?.();
        server.close(() => resolve());
      }),
    tokensIssued: () => issued,
    url: `http://localhost:${port}`,
  };
}
```

Task 15: the client (src/oauth-client.ts)

```
import { APIRequestContext, APIResponse } from '@playwright/test';

interface TokenResponse {
  access_token: string;
  token_type: string;
  expires_in: number;   // seconds
}

export class OAuthClient {
  private token?: string;
  private expiresAt = 0;   // milliseconds

  constructor(
    private request: APIRequestContext,
    private baseUrl: string,
    private clientId = 'my-client',
    private clientSecret = 'my-secret',
  ) {}

  // Raw call: tests use it to check failures such as a wrong secret
  requestToken(): Promise<APIResponse> {
    return this.request.post(`${this.baseUrl}/oauth/token`, {
      form: {   // URL-encoded body, which is what OAuth token endpoints
expect
        grant_type: 'client_credentials',
        client_id: this.clientId,
        client_secret: this.clientSecret,
      },
    });
  }

  // Setup helper: reuse a valid token, otherwise fetch a new one
  async getToken(): Promise<string> {
    if (this.token && Date.now() < this.expiresAt) {
      return this.token;
    }

    const response = await this.requestToken();
    if (!response.ok()) {
      throw new Error(`Token request failed: ${response.status()} ${await
response.text()}`);
```

```
    }

    const body: TokenResponse = await response.json();
    this.token = body.access_token;
    this.expiresAt = Date.now() + body.expires_in * 1000 - 1000;   // seconds
to ms, minus a 1 second margin
    return this.token;
  }

  async getData(): Promise<APIResponse> {
    return this.request.get(`${this.baseUrl}/api/data`, {
      headers: { Authorization: `Bearer ${await this.getToken()}` },
    });
  }
}
```

Task 15: the tests (tests/auth.oauth.spec.ts)

Adjust the import path to wherever your client file lives.

```
import { test, expect } from '@playwright/test';
import { createMockOAuth } from './helpers/mock-oauth-server';
import { OAuthClient } from '../src/oauth-client';

test.describe.configure({ mode: 'serial' });   // one worker, so the port is
not fought over

const mock = createMockOAuth(4010, 3);   // tokens live 3 seconds

test.beforeAll(async () => { await mock.start(); });
test.afterAll(async () => { await mock.stop(); });

test('valid token request returns a bearer token', async ({ request }) => {
  const client = new OAuthClient(request, mock.url);
  const response = await client.requestToken();
  expect(response.status(), await response.text()).toBe(200);

  const body = await response.json();
  expect(typeof body.access_token).toBe('string');
  expect(body.token_type).toBe('Bearer');
  expect(typeof body.expires_in).toBe('number');
});
```

```
test('token is reused until it expires', async ({ request }) => {
  const client = new OAuthClient(request, mock.url);
  const before = mock.tokensIssued();   // the counter is shared across tests

  for (let i = 0; i < 3; i++) {
    const response = await client.getData();
    expect(response.status(), await response.text()).toBe(200);
  }
  expect(mock.tokensIssued() - before).toBe(1);

  await new Promise((resolve) => setTimeout(resolve, 4000));   // longer than
the 3 second lifetime

  await client.getData();
  expect(mock.tokensIssued() - before).toBe(2);
});

test('wrong client secret is rejected', async ({ request }) => {
  const client = new OAuthClient(request, mock.url, 'my-client', 'wrong');
  const response = await client.requestToken();
  expect(response.status(), await response.text()).toBe(401);
  expect((await response.json()).error).toBe('invalid_client');
});

test('GET /api/data without a token is rejected', async ({ request }) => {
  const response = await request.get(`${mock.url}/api/data`);
  expect(response.status(), await response.text()).toBe(401);
});
```

Break it once: set the mock lifetime to 10 seconds, `createMockOAuth(4010, 10)`, and the expiry test fails. Then change `form:` to `data:` in `requestToken()` and the token request fails with `unsupported_grant_type`.

Mistakes and lessons

Each mistake below happened in practice, and most share one cause: a test that could not fail, or a request that went somewhere unexpected.

Mistake	What happened	Fix
Run labeled `[chromium]`	The request went to JSONPlaceholder, so `/auth` returned 404 with `{}`	Use named projects and confirm the `[booker]` label. Log `response.url()`
`toBe(404)` passing by accident	The wrong server happened to return 404	Check where a request went, not only whether the assertion is green
`toMatchObject(booking)` on the whole body	`booking` sat one level down in `body.booking`	Compare the right level. A diff showing your object under an extra key means the wrong level
`request.post` for a read-back	The server answered `Not Found`, and `.json()` threw a parse error	Use the right method, and assert the status before `.json()`
`test.use` at file level	Credentials leaked into the no-credentials test, which returned 200	Scope `test.use` inside a `describe`, or use a fresh `playwright.request.newContext()`
Assertions commented out	The test passed with nothing checked	Uncomment them. A green tick means nothing without assertions
`"/{id}"` in a client method	The literal text `/{id}` was sent	Use backticks and `${id}`, and include `/booking`
`data: newBooking()` in `create`	The caller's argument was ignored	Send the parameter. An unused parameter is a warning sign
`(await response).json()`	The `await` was on the wrong call, so `body` was a Promise	`await response.json()`
Headers as `{[ {...} ]}`	Extra brackets made a syntax error	`headers` is a plain object of `key: value` pairs

Mistake	What happened	Fix
`REQRES_API_KEY` as a bare name	Not a variable, because `.env` values live on `process.env`	Read `process.env.REQRES_API_KEY`
`import "process"` and `npm install process`	Unneeded, because `process` is built into Node	Remove both
Keyless request in a "valid key" test	It passed without the key, a false positive	Assert a field that differs, such as `_meta.context`
`toBe("string")` on a token	Compared the token with the word "string"	`typeof body.accessToken` is `'string'`
Asserting the login body after `/auth/me`	The check passed regardless of the second call	Parse and assert the second response
Credentials taken to be in the URL	`/basic-auth/user/pass` only sets the expected values	The header carries what is sent

Lessons to carry forward

1. Print the status and body before asserting, then assert what the API actually returns.
2. Break every new test once on purpose to prove it can fail.
3. When several assertions fail after one broken line, fix the first error. The rest are often fallout.
4. Document observed behavior against the standard, and raise the gap.
5. Never put keys or tokens in code, chat, screenshots or logs.

Interview Q&A

Short answers you can say out loud, based on what Week 2 covered.

What is the difference between 401 and 403? 401 Unauthorized means the request has no valid credentials, so the client has not proved who it is. 403 Forbidden means the server knows who the client is, or refuses regardless, and denies the action. Restful-Booker returns 403 for a missing token, which I would log as a deviation from the standard 401.

What would you do when an API returns 200 for a failed login? Assert the observed behavior so the suite stays stable, then raise it as a defect. A 200 with an error in the body is a design flaw, because clients and monitoring that trust the status treat it as a success.

How do you test authentication against a shared public API? Each test creates its own data, never hardcodes tokens or IDs, and cleans up with a valid token in `afterEach` or `finally`. After a rejected request, I confirm the data is unchanged. After a delete, I confirm a follow-up GET returns 404.

How do you structure API test automation? Types for payloads and responses, a client class that returns raw responses and attaches auth, a factory for test data, and shared setup and cleanup in hooks or fixtures. Assertions stay in the tests, so negative cases can inspect failures.

Compare API key, Basic, JWT and OAuth client credentials. An API key identifies an application with a static secret. Basic sends a Base64 user and password with every request. A JWT is a signed token with claims and an expiry, obtained by logging in. OAuth client credentials gets a short-lived token for an application from a token endpoint, then sends it as a Bearer header.

Is Basic auth secure? Only over HTTPS. The header is Base64-encoded, not encrypted, so anyone who sees it can decode the password with one line of code.

What is inside a JWT, and what can a tester check? Three dot-separated parts: header, payload and signature. The payload is readable, so I can decode it and assert claims such as `exp` and `iat`. A tampered signature should be rejected by the server. Secrets should never be in the payload.

How do you handle token expiry in tests? Cache the token with its expiry time, reuse it while valid, and fetch a new one when it has expired, with a small safety margin. Test both reuse and refetch, and keep time units straight, because `Date.now()` is in milliseconds and `expires_in` is in seconds.

What is the difference between `data` and `form` in a request? `data` sends a JSON body. `form` sends URL-encoded `key=value` pairs, which is what an HTML form and OAuth token endpoints use. Sending JSON to an endpoint that expects a form fails because the

server cannot find the fields.

How do you keep secrets out of test code? Store them in a `.env` file loaded with `dotenv`, list `.env` in `.gitignore`, use CI secrets in pipelines, and never print them. Skip tests with a clear message when a required variable is missing.

How do you make sure a negative test is meaningful? Print the real response first, then assert the exact status, the error field and that no sensitive data is returned. Break the test once on purpose, and check it can detect the problem it is meant to catch.

Self-check and next steps

Tick each item only when you can do it from memory, without looking at the code.

- [ ] I can explain why a run labeled `[chromium]` means the wrong `baseURL`
- [ ] I can write a create, read, update, delete test with a token and a read-back
- [ ] I can explain why `body.booking` and `body` are different levels
- [ ] I can explain 401 vs 403 and why 200 for a failed login is a defect
- [ ] I know why `test.use` at file level leaks into other tests
- [ ] I can write interfaces whose names match the JSON keys exactly
- [ ] I can explain why the client returns raw responses and only setup throws
- [ ] I can explain why `request` is created in `beforeEach` and not at file level
- [ ] I can load a key from `.env` and keep it out of code, chat and logs
- [ ] I can explain why a keyless request made the first API key test a false positive
- [ ] I can build a Basic auth header and explain why it is not encryption
- [ ] I can name the three parts of a JWT and say which one is readable
- [ ] I can explain `form` vs `data` and why OAuth token requests use `form`
- [ ] I break every new test once on purpose to prove it can fail

Still pending from Week 2

- [ ] Run the Task K file and confirm all tests pass
- [ ] Task A: remove `.skip`, use `typeof` plus a length check, and run it
- [ ] Task 9: data-driven wrong credentials, three sets
- [ ] Task 12: run all three tests together and do the break-it-once
- [ ] Task 13: add the commented assertions back, scope `test.use`, and do step 5
- [ ] Task 14: read the `/auth/me` body, then steps 3 to 6

- [ ] Task 15: write the `OAuthClient` tests against the mock server
- [ ] Clean up: remove `console.log` and `.skip`, check `.env` is in `.gitignore`, run `npm uninstall process`

Next: Week 3. Custom fixtures to replace the `let client` pattern, schema validation with `zod`, test tags and annotations, and running in CI with the HTML report.